

FACULDADE DE TECNOLOGIA DE SÃO JOSÉ DOS CAMPOS
FATEC PROFESSOR JESSEN VIDAL

Artur Mota, Cauê Silva, Gustavo Araujo, Jonathan Alvarenga, Luiz Vinicius,
Maria Eduarda, Wanderson Oliveira

São José dos Campos

2026

Sumário

1. Introdução.....	
1.1 Contextualização.....	
1.2 Problema	
1.3 Justificativa	
1.4 Objetivo geral	
 2. Desenvolvimento.....	
3. Resultados.....	
4. Referências.....	

1. Introdução

A análise de dados aplicada à logística tem papel importante na compreensão de ocorrências relacionadas ao transporte e a circulação de veículos, permitindo organizar informações e identificar características relevantes para apoiar análises e processos de tomada de decisão. Neste contexto, este projeto tem como foco o tratamento e a organização de dados relacionados a sinistros de veículos no Estado de São Paulo, considerando informações referentes ao período de 2015 a 2025.

A primeira etapa do projeto, denominada Sprint 1, foi destinada à preparação, tratamento, padronização e validação das bases de dados que serão utilizadas nas etapas posteriores. Para isso, foram trabalhadas três bases principais: sinistros, população e óbitos. O tratamento foi realizado utilizando Python, por meio do Google Colab e da biblioteca Pandas, possibilitando a manipulação, consolidação e verificação dos dados.

1.1 Contextualização

O projeto está inserido no contexto da análise de dados aplicada à logística e ao transporte de veículos no Estado de São Paulo. Para a realização das análises propostas, é necessário trabalhar com diferentes conjuntos de informações referentes aos municípios e ao período de 2015 a 2025, possibilitando a organização a organização dos dados relacionados aos sinistros, à população e aos óbitos.

Nesse cenário, a Sprint 1 concentra-se na preparação das bases de dados que serão utilizadas nas etapas posteriores do projeto.

1.2 Problema

O projeto parte da necessidade de organizar e preparar diferentes bases de dados relacionadas aos sinistros de veículos no Estado de São Paulo, considerando informações referentes ao período de 2015 a 2025. Antes da realização das análises, os dados precisam passar por processos de verificação, limpeza e

padronização, uma vez que diferentes bases podem apresentar estruturas, formatos e informações que necessitam de tratamento.

Durante a análise inicial da base de sinistros, foi identificada uma inconsistência relacionada à importação dos dados, com registros que apresentavam campos agrupados de forma inadequada. Esse problema exigiu a identificação dos registros afetados, a separação dos campos, a reconstrução das informações e o reposicionamento dos valores nas colunas correspondentes. Além disso, foi necessário verificar a existência de registros duplicados, valores nulos e inconsistências na estrutura das bases. Na base populacional, foram realizadas verificações de duplicidades, anos disponíveis, códigos municipais e correspondência entre municípios e Unidades Federativas. Na base de óbitos, também foram analisados valores nulos, registros duplicados e informações sem localização de município ou UF.

1.3 Justificativa

A realização da Sprint 1 justifica-se pela necessidade de garantir que os dados utilizados no projeto estejam organizados, padronizados e validados antes de serem utilizados nas etapas posteriores de análise. Como o projeto trabalha com diferentes bases de informações referentes ao Estado de São Paulo e ao período de 2015 a 2025, torna-se necessário verificar a estrutura e a qualidade dos dados para reduzir possíveis inconsistências durante sua utilização.

Na base de sinistros, o tratamento foi necessário para consolidar os arquivos anuais, remover duplicidades, corrigir problemas relacionados à estrutura dos registros e organizar informações referentes a municípios, UF, tipos de veículos, motocicletas, datas e vítimas. A base de população também precisou ser validada e padronizada, incluindo a verificação dos anos, municípios, códigos municipais, duplicidades e valores nulos. Esse processo possibilitou a geração de uma base final validada para utilização nas etapas seguintes do projeto.

1.4 Objetivo geral

O objetivo geral da Sprint 1 é **preparar, tratar, padronizar e validar as bases de dados de sinistros, população e óbitos do estado de São Paulo,**

abrangendo o período de 2015 a 2025, de modo a garantir que as informações estejam organizadas, consistentes e adequadas para utilização nas etapas posteriores do projeto.

2. Desenvolvimento (Sprint 1)

O tratamento dos dados foi realizado utilizando **Python, Pandas e Google Colab**, com os arquivos armazenados e acessados por meio do Google Drive. As atividades foram organizadas de forma sequencial, permitindo identificar inconsistências, realizar as correções necessárias e gerar as bases tratadas para utilização nas próximas etapas.

2.1 Preparação do ambiente

Inicialmente, foi preparado o ambiente de desenvolvimento no Google Colab. Para isso, foi realizada a importação da biblioteca Pandas e a conexão do Google Drive ao ambiente, permitindo o acesso aos arquivos utilizados no tratamento dos dados.

```
[59]
✓ 9s

coluna_errada = df_sinistros.columns[0]

df_erro = df_sinistros[
    df_sinistros[coluna_errada].notna()
].copy()

df_correto = df_sinistros[
    df_sinistros[coluna_errada].isna()
].copy()

linhas_recuperadas = df_erro[coluna_errada].apply(
    lambda x: next(csv.reader([str(x)]))
)

print("Quantidade de campos encontrados:")

print(
    linhas_recuperadas.apply(len)
    .value_counts()
    .sort_index()
```

2.2 Tratamento da base de sinistros

- A primeira base trabalhada foi a de **sinistros de veículos**. Inicialmente, foram definidos os caminhos dos arquivos CSV correspondentes aos anos de 2015 a 2025 e realizada a leitura dos arquivos utilizando o Pandas.

```

arquivo_1 = '/content/drive/MyDrive/Data/sinistros/acidentes2015.csv'
arquivo_2 = '/content/drive/MyDrive/Data/sinistros/acidentes2016.csv'
arquivo_3 = '/content/drive/MyDrive/Data/sinistros/acidentes2017.csv'
arquivo_4 = '/content/drive/MyDrive/Data/sinistros/acidentes2018.csv'
arquivo_5 = '/content/drive/MyDrive/Data/sinistros/acidentes2019.csv'
arquivo_6 = '/content/drive/MyDrive/Data/sinistros/acidentes2020.csv'
arquivo_7 = '/content/drive/MyDrive/Data/sinistros/acidentes2021.csv'
arquivo_8 = '/content/drive/MyDrive/Data/sinistros/acidentes2022.csv'
arquivo_9 = '/content/drive/MyDrive/Data/sinistros/acidentes2023.csv'
arquivo_10 = '/content/drive/MyDrive/Data/sinistros/acidentes2024.csv'
arquivo_11 = '/content/drive/MyDrive/Data/sinistros/acidentes2025.csv'

sinistros15 = pd.read_csv(arquivo_1, low_memory=False, sep=';', encoding='latin1')
sinistros16 = pd.read_csv(arquivo_2, low_memory=False, sep=';', encoding='latin1')
sinistros17 = pd.read_csv(arquivo_3, low_memory=False, sep=';', encoding='latin1')
sinistros18 = pd.read_csv(arquivo_4, low_memory=False, sep=';', encoding='latin1')
sinistros19 = pd.read_csv(arquivo_5, low_memory=False, sep=';', encoding='latin1')
sinistros20 = pd.read_csv(arquivo_6, low_memory=False, sep=';', encoding='latin1')
sinistros21 = pd.read_csv(arquivo_7, low_memory=False, sep=';', encoding='latin1')
sinistros22 = pd.read_csv(arquivo_8, low_memory=False, sep=';', encoding='latin1')
sinistros23 = pd.read_csv(arquivo_9, low_memory=False, sep=';', encoding='latin1')
sinistros24 = pd.read_csv(arquivo_10, low_memory=False, sep=';', encoding='latin1')
sinistros25 = pd.read_csv(arquivo_11, low_memory=False, sep=';', encoding='latin1')

```

- Após a importação, os arquivos anuais foram consolidados em uma única base. Nessa etapa, foram removidas linhas duplicadas e realizadas padronizações nos nomes das colunas, textos e representações de valores ausentes.

```

[42]
✓ 2s      sinistros21_25 = pd.concat([sinistros15, sinistros16, sinistros17, sinistros18, sinistros19, sinistros20, sinistros21, sinistros22, sinistros23, sinis

[43]
✓ 5s      df_sinistros = sinistros21_25.copy()

[44]
✓ 9s      df_sinistros = df_sinistros.drop_duplicates()

[46]
✓ 0s      df_sinistros.columns = df_sinistros.columns.str.strip()

[47]
✓ 0s      df_sinistros.columns = (df_sinistros.columns.str.upper().str.strip().str.replace(' ', '_'))

[48]
✓ 14s     for coluna in df_sinistros.select_dtypes(include='object').columns:df_sinistros[coluna] = df_sinistros[coluna].str.strip()

[49]
✓ 18s     df_sinistros = df_sinistros.replace(['', ' ', 'NULL', 'null', 'N/A', 'NA'], pd.NA)

[51]
✓ 0s      print("Linhas:", df_sinistros.shape[0])
          print("Colunas:", df_sinistros.shape[1])

✓      Linhas: 2139368
          Colunas: 38

```

- Em seguida, foi realizado um diagnóstico da base consolidada, verificando a quantidade de linhas e colunas, registros duplicados e valores nulos. Essa análise também permitiu identificar uma inconsistência relacionada à estrutura de uma das colunas da base.

```
[52] print("Linhas:", df_sinistros.shape[0])
✓ 13s print("Colunas:", df_sinistros.shape[1])
print("Duplicados:", df_sinistros.duplicated().sum())
print("Valores nulos:", df_sinistros.isnull().sum().sum())

Linhas: 2139368
Colunas: 38
Duplicados: 0
Valores nulos: 18215981

[57]
✓ 0s coluna_errada = df_sinistros.columns[0]

print("Nome da coluna incorreta:")
print(coluna_errada)

print("\nRegistros com problema:")
print(df_sinistros[coluna_errada].notna().sum())

print("\nRegistros com estrutura correta:")
print(df_sinistros[coluna_errada].isna().sum())
```

- Para solucionar o problema identificado, foram isolados os registros afetados e realizada a leitura dos campos utilizando o módulo CSV. Posteriormente, os campos agrupados foram separados e reorganizados.

```
df_backup = df_sinistros.copy()

coluna_errada = df_sinistros.columns[0]

colunas_corretas = df_sinistros.columns[1:29].tolist()

df_recuperado = pd.DataFrame(
    linhas_recuperadas.tolist(),
    index=linhas_recuperadas.index,
    columns=colunas_corretas
)

display(df_recuperado.head(10))
```

```
[59]
✓ 9s coluna_errada = df_sinistros.columns[0]

df_erro = df_sinistros[
    df_sinistros[coluna_errada].notna()
].copy()

df_correto = df_sinistros[
    df_sinistros[coluna_errada].isna()
].copy()

linhas_recuperadas = df_erro[coluna_errada].apply(
    lambda x: next(csv.reader([str(x)]))
)

print("Quantidade de campos encontrados:")

print(
    linhas_recuperadas.apply(len)
    .value_counts()
    .sort_index()
```

```
df_sinistros_corrigido = df_sinistros.copy()

df_sinistros_corrigido.loc[
    df_recuperado.index,
    colunas_corretas
] = df_recuperado

df_sinistros_corrigido = df_sinistros_corrigido.drop(
    columns=[coluna_errada]
)

display(df_sinistros_corrigido.head(10))
```

Após a reconstrução, os valores recuperados foram reposicionados nas respectivas colunas e a coluna que apresentava a estrutura incorreta foi removida. Dessa maneira, a base consolidada passou a apresentar os registros em uma estrutura adequada para as etapas seguintes.

2.3 Seleção e organização dos veículos pesados

Após a correção da base, foi realizado um filtro para selecionar os registros referentes aos veículos considerados pesados. Foram utilizadas as categorias **Caminhão, Caminhão-trator, Ônibus e Micro-ônibus**, sendo realizada a conferência da quantidade de registros por categoria.

```
veiculos_pesados = [  
    'CAMINHÃO',  
    'CAMINHÃO-TRATOR',  
    'CAMINHÃO TRATOR',  
    'ÔNIBUS',  
    'MICRO-ÔNIBUS']
```

```
print("Total de registros:", len(df_pesados))  
  
print("\nTipos de veículos selecionados:")  
  
print(df_pesados['TIPO_VEICULO'].value_counts())
```

Total de registros: 388394

Tipos de veículos selecionados:

TIPO_VEICULO	
Caminhão	139721
Caminhão-trator	116705
Ônibus	77043
Caminhão-Trator	42514
Micro-ônibus	12411

Name: count, dtype: int64

```
[77]  
✓ 1s df_pesados = df_pesados.sort_values(  
    by=['UF', 'MUNICIPIO']  
    ).reset_index(drop=True)  
  
display(df_pesados)
```

- Na sequência, os registros selecionados foram organizados por **UF e município**, facilitando a estruturação das informações geográficas. Também foram realizados tratamentos nos campos numéricos, incluindo a conversão de determinados identificadores e campos para valores inteiros.

```
[82]
✓ 1s
colunas = [
    'ID',
    'PESID',
    'BR',
    'ID_VEICULO',
    'IDADE',
    'ANO_FABRICACAO_VEICULO'
]

for coluna in colunas:
    df_pesados[coluna] = (
        pd.to_numeric(
            df_pesados[coluna],
            errors='coerce'
        ).astype('Int64')
    )

[84]
✓ 0s
df_pesados['TOTAL_FERIDOS'] = (
    pd.to_numeric(df_pesados['FERIDOS_LEVES'], errors='coerce').fillna(0) +
    pd.to_numeric(df_pesados['FERIDOS_GRAVES'], errors='coerce').fillna(0)
).astype(int)
```

2.4 Identificação de sinistros envolvendo motocicletas

- Outra etapa do tratamento consistiu na identificação dos sinistros que envolveram motocicletas. Para isso, foram identificados os IDs associados ao tipo de veículo “MOTOCICLETA” e, posteriormente, criada a variável **ENVOLVEU_MOTOCICLETA**, permitindo indicar se determinado registro possuía ou não envolvimento desse tipo de veículo.

```
[90]
✓ 2s
ids_motos = df_sinistros_corrigido.loc[
    df_sinistros_corrigido['TIPO_VEICULO']
    .astype('string')
    .str.strip()
    .str.upper()
    .eq('MOTOCICLETA'),
    'ID'
].dropna().astype('string').str.strip().str.replace(
    r'\.0$', '', regex=True
).unique()

[91]
✓ 0s
df_pesados['ENVOLVEU_MOTOCICLETA'] = (
    df_pesados['ID']
    .astype('string')
    .str.strip()
    .str.replace(r'\.0$', '', regex=True)
    .isin(ids_motos)
    .map({True: 'SIM', False: 'NÃO'})
)
```

```
[93]
✓ 4s
print("Total de registros:", len(df_pesados))

print("Total de colunas:", df_pesados.shape[1])

print("Registros duplicados:",
      df_pesados.duplicated().sum())

print("Total de valores ausentes:",
      df_pesados.isnull().sum().sum())

print("Sinistros distintos:",
      df_pesados['ID'].nunique())
```

Total de registros: 388394
 Total de colunas: 40
 Registros duplicados: 0
 Total de valores ausentes: 1593202
 Sinistros distintos: 251482

2.5 Verificação das datas e dos tipos de veículos

- As datas dos sinistros foram convertidas para o formato de data e verificadas quanto à existência de valores inválidos ou ausentes. A análise indica que não foram identificadas datas inválidas ou ausentes, com data mínima em **01/01/2015** e data máxima em **31/12/2025**.

```
[94]
✓ Os
datas = pd.to_datetime(
    df_pesados['DATA_INVERSA'],
    errors='coerce',
    dayfirst=True,
    format='mixed'
)

print("Datas inválidas ou ausentes:", datas.isna().sum())

print("Data mais antiga:", datas.min())

print("Data mais recente:", datas.max())

---
Datas inválidas ou ausentes: 0
Data mais antiga: 2015-01-01 00:00:00
Data mais recente: 2025-12-31 00:00:00
```

- Também foi realizada a contagem das categorias de veículos presentes na tabela tratada. A evidência apresenta as quantidades para caminhões, caminhões-tratores, ônibus e micro-ônibus.

```
[95]
✓ Os
display(
    df_pesados['TIPO_VEICULO']
    .value_counts(dropna=False)
)

---
count
TIPO_VEICULO
Caminhão      139721
Caminhão-trator  116705
Ônibus        77043
Caminhão-Trator  42514
Micro-ônibus   12411

dtype: int64
```

- Posteriormente, foram verificadas as informações geográficas, considerando UF e município, além dos campos relacionados às vítimas. Foram avaliados valores mínimos, máximos, negativos e ausentes para as variáveis de vítimas.

```
print(
    "Estados sem informação:",
    df_pesados['UF'].isna().sum()
)

print(
    "Municípios sem informação:",
    df_pesados['MUNICIPIO'].isna().sum()
)

print("\nEstados presentes na tabela:")

display(
    df_pesados['UF']
    .value_counts(dropna=False)
)
```

```

colunas_vitimas = [
    'ILESOS',
    'FERIDOS_LEVES',
    'FERIDOS_GRAVES',
    'MORTOS'
]

for coluna in colunas_vitimas:

    valores = pd.to_numeric(
        df_pesados[coluna],
        errors='coerce'
    )

    print(f"\n{coluna}")

    print("Valor mínimo:", valores.min())

    print("Valor máximo:", valores.max())

    print("Valores negativos:", (valores < 0).sum())

    print("Valores ausentes:", valores.isna().sum())

```

- Ao final do tratamento da base de sinistros, as colunas relacionadas às vítimas e a coluna **TOTAL_FERIDOS** foram removidas, sendo gerado o arquivo CSV da tabela tratada.

```

df_pesados = df_pesados.drop(
    columns=[
        'ILESOS',
        'FERIDOS_LEVES',
        'FERIDOS_GRAVES',
        'MORTOS'
    ]
)

df_pesados = df_pesados.drop(
    columns=['TOTAL_FERIDOS'],
    errors='ignore'
)

from google.colab import files

df_pesados.to_csv(
    'df_pesados_final.csv',
    index=False,
    sep=';',
    encoding='utf-8-sig'
)

```

2.6 Tratamento da base de população

- A segunda base trabalhada foi a de **população**. Inicialmente, foram importadas as bibliotecas necessárias, realizada a conexão com o Google Drive e definido o caminho do arquivo populacional utilizado no tratamento.

```

import pandas as pd
from google.colab import files

from google.colab import drive, files
import os

drive.mount('/content/drive')

Mounted at /content/drive

caminho = '/content/drive/MyDrive/Data/sinistros/populacao_municipios_2015_2025_tratada.csv'

```

- A base foi importada em formato CSV, com definição do separador e da codificação, mantendo o código do município como texto para evitar alterações em sua representação.

```
import pandas as pd
from google.colab import drive

drive.mount('/content/drive')

caminho = '/content/drive/MyDrive/Data/sinistros/populacao_municipios_2015_2025_tratada.csv'

df_pop = pd.read_csv(
    caminho,
    sep=',',
    encoding='utf-8-sig',
    dtype={'COD_MUNICIPIO': str}
)

print("Tabela importada com sucesso!")
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Tabela importada com sucesso!

- Na sequência, foram realizadas verificações iniciais da estrutura da tabela, incluindo visualização dos primeiros registros, dimensões, nomes das colunas e tipos de dados.

```
display(df_pop.head(10))

print("Dimensões da tabela:", df_pop.shape)

print("\nColunas:")
print(df_pop.columns.tolist())

print("\nInformações gerais:")
df_pop.info()
```

	ANO	COD_MUNICIPIO	MUNICIPIO	UF	POPULACAO
0	2015	120001	ACRELÂNDIA	AC	13786.0
1	2015	120005	ASSIS BRASIL	AC	7124.0
2	2015	120010	BRASILEIA	AC	24352.0
3	2015	120013	BUJARI	AC	10415.0
4	2015	120017	CAPIXABA	AC	9913.0
5	2015	120020	CRUZEIRO DO SUL	AC	90107.0
6	2015	120025	EPITACIOLÂNDIA	AC	17245.0
7	2015	120030	FEIJÓ	AC	36415.0

- Foi realizada ainda a verificação da distribuição dos registros por ano e da quantidade de municípios distintos, com conferência da presença dos anos de **2015 a 2025**.

```
registros_ano = (
    df_pop.groupby("ANO")
    .agg(
        TOTAL_REGISTROS=("COD_MUNICIPIO", "size"),
        MUNICIPIOS_DISTINTOS=("COD_MUNICIPIO", "nunique"),
        POPULACAO_AUSENTE=("POPULACAO", lambda x: x.isna().sum())
    )
    .reset_index()
)

anos_esperados = set(range(2015, 2026))

anos_encontrados = set(df_pop["ANO"].dropna().unique())

anos_faltantes = sorted(
    anos_esperados - anos_encontrados
)

print("REGISTROS POR ANO:")
display(registros_ano)

print("\nANOS FALTANTES:")
print(anos_faltantes)

print("\nTOTAL DE MUNICÍPIOS DISTINTOS:")
print(df_pop["COD_MUNICIPIO"].nunique())
```

- Posteriormente, foram verificadas duplicidades por município e ano e a quantidade de anos disponíveis para cada município.

```

duplicados = df_pop[
    df_pop.duplicated(
        subset=["COD_MUNICIPIO", "ANO"],
        keep=False
    )
]

print("REGISTROS DUPLICADOS:")
print(len(duplicados))

display(duplicados.head(20))

anos_municipio = (
    df_pop.groupby(["COD_MUNICIPIO", "MUNICIPIO", "UF"])
    ["ANO"]
    .nunique()
    .reset_index(name="TOTAL_ANOS")
)

municipios_incompletos = anos_municipio[
    anos_municipio["TOTAL_ANOS"] < 11
]

print("\nMUNICÍPIOS COM ANOS FALTANTES:")
print(len(municipios_incompletos))

display(municipios_incompletos.head(30))

```

- Na etapa de padronização, foram ajustados os nomes das colunas e os tipos dos campos de população e ano. O código municipal também foi normalizado, mantendo sua estrutura como texto.

```

df_pop_limpo.columns = (
    df_pop_limpo.columns
    .str.strip()
    .str.upper()
)

df_pop_limpo["POPULACAO"] = (
    df_pop_limpo["POPULACAO"]
    .astype("Int64")
)

df_pop_limpo["ANO"] = (
    df_pop_limpo["ANO"]
    .astype("Int64")
)

df_pop_limpo["COD_MUNICIPIO"] = (
    df_pop_limpo["COD_MUNICIPIO"]
    .astype("string")
    .str.strip()
    .str.zfill(6)
)

df_pop_limpo["MUNICIPIO"] = (
    df_pop_limpo["MUNICIPIO"]
    .astype("string")

```

- Os nomes dos municípios e as siglas das unidades federativas foram padronizados, seguida da conferência dos respectivos tipos de dados.

```

df_pop_limpo["MUNICIPIO"] = (
    df_pop_limpo["MUNICIPIO"]
    .astype("string")
    .str.strip()
    .str.upper()
)

df_pop_limpo["UF"] = (
    df_pop_limpo["UF"]
    .astype("string")
    .str.strip()
    .str.upper()
)

print("PADRONIZAÇÃO CONCLUÍDA!")

print("\nTIPOS DE DADOS:")
print(df_pop_limpo.dtypes)

print("\nPRIMEIROS REGISTROS:")
display(df_pop_limpo.head(10))

```

```

PADRONIZAÇÃO CONCLUÍDA!

TIPOS DE DADOS:
ANO                Int64
COD_MUNICIPIO      string[python]
MUNICIPIO          string[python]
...

```

- Também foi realizada a validação dos códigos municipais, verificando o formato dos códigos e possíveis divergências entre município e UF associados a um mesmo código.

```

codigos_invalidos = df_pop_limpo[
    ~df_pop_limpo["COD_MUNICIPIO"].str.fullmatch(
        r"\d{6}",
        na=False
    )
]

print("CÓDIGOS MUNICIPAIS COM FORMATO INVÁLIDO:")
print(len(codigos_invalidos))

display(codigos_invalidos)

municipios_divergentes = (
    df_pop_limpo.groupby("COD_MUNICIPIO")
    .agg(
        NOMES_DISTINTOS=("MUNICIPIO", "nunique"),
        UFS_DISTINTAS=("UF", "nunique")
    )
    .query("NOMES_DISTINTOS > 1 or UFS_DISTINTAS > 1")
)

print("\nCÓDIGOS COM NOMES OU UFS DIVERGENTES:")
print(len(municipios_divergentes))

display(municipios_divergentes)

```

```

CÓDIGOS MUNICIPAIS COM FORMATO INVÁLIDO:
0

```

- Ao final, foi elaborado um relatório de qualidade contendo verificações de duplicidades, combinações esperadas de município e ano e valores nulos.

```

duplicados = df_pop_limpo.duplicated(
    subset=["COD_MUNICIPIO", "ANO"],
    keep=False
)

total_duplicados = duplicados.sum()

municipios = df_pop_limpo["COD_MUNICIPIO"].dropna().unique()

anos_esperados = range(2015, 2026)

grade = pd.MultiIndex.from_product(
    [municipios, anos_esperados],
    names=["COD_MUNICIPIO", "ANO"]
)

chaves_existentes = pd.MultiIndex.from_frame(
    df_pop_limpo[["COD_MUNICIPIO", "ANO"]]
    .drop_duplicates()
    .dropna()
)

anos_ausentes = grade.difference(chaves_existentes)

nulos = df_pop_limpo.isna().sum()

print("RELATÓRIO FINAL DE QUALIDADE")
print(
    "Municípios distintos:",
    df_pop_limpo["COD_MUNICIPIO"].nunique()
)

print(
    "Anos distintos:",
    df_pop_limpo["ANO"].nunique()
)

print(
    "Registros envolvidos em duplicatas:",
    total_duplicados
)

print(
    "Combinações município/ano ausentes:",
    len(anos_ausentes)
)

print("\nVALORES NULOS POR COLUNA:")
print(nulos)

print("\nCOMBINAÇÕES MUNICÍPIO/ANO AUSENTES:")
display(anos_ausentes.to_frame(index=False).head(20))

... RELATÓRIO FINAL DE QUALIDADE
-----
Total de registros: 61281
..

```

```

28] from google.colab import files
    df_pop_limpo.to_csv(
        "POPULACAO_BR_2015_2025_FINAL.csv",
        index=False,
        encoding="utf-8-sig"
    )

    files.download("POPULACAO_BR_2015_2025_FINAL.csv")

```

2.7 Tratamento da base de óbitos

- A terceira base tratada foi a de **óbitos**. O processo começou com a importação do arquivo CSV no Google Colab utilizando Pandas, com separador por ponto e vírgula, codificação UTF-8 e preservação do código do município como texto.

```

import pandas as pd
from google.colab import files

uploaded = files.upload()

arquivo = next(iter(uploaded))

df_obitos = pd.read_csv(
    arquivo,
    sep=";",
    encoding="utf-8-sig",
    dtype={"COD_MUNICIPIO": "string"},
    low_memory=False
)

```

- Antes da realização das validações, foi criada uma cópia da base original. Em seguida, foram verificadas as dimensões da tabela e os nomes das colunas.

```

df_obitos_original = df_obitos.copy()

print("IMPORTAÇÃO DA BASE DE ÓBITOS")
print("-" * 40)

print("Dimensões:", df_obitos.shape)

print("\nCOLUMNAS:")
print(df_obitos.columns.tolist())

```

- Posteriormente, foram analisados os tipos de dados, os primeiros registros e a quantidade de valores nulos por coluna. Também foram identificadas possíveis duplicidades na combinação de **ANO** e **COD_MUNICIPIO**.

```

print("\nTIPOS DE DADOS:")
df_obitos.info()

print("\nPRIMEIROS REGISTROS:")
display(df_obitos.head(10))

nulos = df_obitos.isna().sum()

print("VALORES NULOS POR COLUNA:")
display(nulos.to_frame("TOTAL_NULOS"))

duplicados = df_obitos.duplicated(
    subset=["ANO", "COD_MUNICIPIO"],
    keep=False
)

```

- Na sequência, foram analisados os registros duplicados e os registros que não apresentavam informações de município ou UF, permitindo identificar informações sem localização para avaliação posterior.

```

duplicados = df_obitos.duplicated(
    subset=["ANO", "COD_MUNICIPIO"],
    keep=False
)

print("\nREGISTROS ENVOLVIDOS EM DUPLICATAS:")
print(duplicados.sum())

registros_sem_localizacao = df_obitos[
    df_obitos["MUNICIPIO"].isna() |
    df_obitos["UF"].isna()
]

print("\nREGISTROS SEM MUNICÍPIO OU UF:")
print(len(registros_sem_localizacao))

display(registros_sem_localizacao.head(20))

```

- Por fim, os dados foram agrupados por ano para verificar a quantidade de registros, os códigos municipais distintos e o total de óbitos em cada período, além do cálculo do total geral da base.

```
resumo_anual = (  
    df_obitos.groupby("ANO")  
    .agg(  
        TOTAL_REGISTROS=("COD_MUNICIPIO", "size"),  
        CODIGOS_DISTINTOS=("COD_MUNICIPIO", "nunique"),  
        TOTAL_OBITOS=("OBITOS", "sum")  
    )  
    .reset_index()  
)  
  
print("DISTRIBUIÇÃO ANUAL DOS ÓBITOS:")  
display(resumo_anual)  
  
print("\nTOTAL GERAL DE ÓBITOS:")  
print(df_obitos["OBITOS"].sum())
```

